MediCheck

AI-Powered Healthcare Bill Accuracy & Dispute Assistant

Custom Project Rubric & Development Guidelines v2 | Microservices on Render | 11 Weeks | 2-Person Team

V3 update to align with Proposal v4

OVERVIEW

MediCheck is an AI-powered web application built as a three-service microservices system deployed on Render, analysing US medical bills and EOBs to detect billing errors, validate charges, and generate dispute letters. The system applies RAG, OCR, NLP, and domain-driven design, consolidating skills from all five prior Quantic MSSE projects into a single capstone submission.

> **Architecture:** Three independent Render services aligned to DDD bounded contexts: Service 1 (React frontend), Service 2 (Bill Analysis API — Bill Processing Domain), Service 3 (RAG & Letter Service — Knowledge & Explanation Domain). Each service is independently deployable with its own URL, environment variables, and responsibility boundary.

LEARNING OUTCOMES

Skill Area	Prior Project	Demonstrated In MediCheck
Requirements, user stories, wireframes	Project 1	15+ user stories (3 personas), user journey, 20+ requirements, Figma wireframes with requirement annotations
React + Flask + PostgreSQL web app	Project 2	Service 1 (React/Vite), Service 2 (Flask/PostgreSQL), Flexbox/Grid layout, form validation
Design patterns, anti-patterns, UML, DDD, microservices	Project 3	Strategy pattern (detectors), Chain of Responsibility (RAG), 2x UML diagrams, bounded context diagram, microservices architecture diagram
ML pipeline, testing, CI/CD gated deployment	Project 4	Medicare rate outlier scoring, pytest unit/integration/smoke tests, GitHub Actions → Render auto-deploy
RAG design, evaluation, ChromaDB, LangChain	Project 5	ChromaDB vector store, LangChain LCEL chain, groundedness + citation accuracy + latency evaluation
Agile Scrum, sprints, collaborative engineering	Capstone	Trello board, sprint demos, PR-based GitHub workflow, Product Owner + Scrum Master roles

PROJECT DESCRIPTION

MediCheck is built across three independently deployed Render services. The following description covers each service and its responsibilities.

Service 1 — React Frontend

A React + Vite SPA deployed as a Render Static Site. Handles all user interaction across four pages: Home/Upload, Field Confirmation, Error Report, and Dispute Letter. Communicates exclusively with Service 2 via REST. Independently deployable — UI changes require no backend restart.

Service 2 — Bill Analysis API (Bill Processing Domain)

A Flask REST API deployed as a Render Web Service. Owns document ingestion via AWS Textract, all four error detection modules (Strategy pattern), PostgreSQL session management, and result storage. Calls Service 3 for RAG explanations and letter drafts. This is the stateful core of the system.

Service 3 — RAG & Letter Generation (Knowledge & Explanation Domain)

A Flask microservice deployed as a second Render Web Service. Owns the ChromaDB vector store, LangChain retrieval chain, LLM prompt engineering, and dispute letter content generation. Receives structured error payloads from Service 2 and returns grounded, cited explanations. Fully stateless — no user session data.

Four Error Detection Modules (Service 2)

- **Module 1:** Module 1 — Duplicate charge detection: same CPT code billed twice on the same date. Structural comparison — no CPT text descriptions required.
- **Module 2:** Module 2 — Medicare rate outlier: billed amount exceeds 300% of CMS Physician Fee Schedule locality rate.
- **Module 3:** Module 3 — EOB reconciliation: cross-reference line items between provider bill and insurer EOB; flag mismatches.
- **Module 4:** Module 4 — No Surprises Act violation: out-of-network provider in a context where federal law (2022) prohibits balance billing.

Design pattern: All four modules implement the Strategy pattern via a BaseDetector abstract class. The ErrorDetectionEngine runs any list of BaseDetector implementations and aggregates results. Document this in the design and testing document with a UML class diagram.

RAG Knowledge Base (Service 3)

- CMS Physician Fee Schedule — Medicare rates by CPT code and locality (free CSV from cms.gov)
- ICD-10-CM coding guidelines — diagnosis coding for upcoding detection (free PDF from cms.gov)
- No Surprises Act text and CMS guidance — federal out-of-network billing protections (free from congress.gov)
- CMS Procedure-to-RVU crosswalk — procedure groupings without AMA-copyrighted descriptions (free Excel from cms.gov)

CPT licensing: Full CPT descriptions are AMA-copyrighted (~$1,000-2,500/yr). All detection modules use CPT code numbers and CMS rate data only. Documented as an explicit architectural decision in the design document.

TEAM ROLES & RESPONSIBILITIES

Role	Member	Primary Accountability

Product Owner + Developer	Member 1	User stories, Trello backlog, acceptance criteria, Figma wireframes. Leads Service 3 entirely: ChromaDB ingestion, LangChain RAG chain, LLM prompts, RAG evaluation, dispute letter generation. Drives presentation narrative. Equal development contribution.
Scrum Master + Code Owner + Developer	Member 2	Sprint facilitation, standups. Owns GitHub repo, approves all PRs. Sets up all three Render services and GitHub Actions CI/CD pipeline. Leads Service 2 entirely: OCR integration, all four detection modules, Flask API, PostgreSQL schema, python-docx/ReportLab export. Equal development contribution.

STEP-BY-STEP DEVELOPMENT INSTRUCTIONS

Sequence: Steps 1-2 in Sprint 1 (process and design — no code). Steps 3-4 in Sprint 2 (infrastructure + OCR). Steps 5-6 in Sprint 3 (knowledge base + error detection). Step 7 in Sprint 4 (RAG explanations + dispute letters — end-to-end flow complete). Steps 8-10 in Sprint 5 (evaluation + documentation + testing — code freeze). Sprint 6 is 1 week — presentation recording only.

Step 1 — Requirements, User Stories & Wireframes [Sprint 1 — Member 1]

- Write 15+ user stories from three personas: patient uploading first bill, patient disputing a known error, developer consuming the API. Standard format: 'As a [role], I want to [action] so that [outcome].'
- Draft one complete user journey for the primary patient flow: upload → confirm → error report → dispute letter. Include decision points and error paths.
- Write 20+ system requirements, numbered and categorised (functional / non-functional). Must be quantifiable. Examples: 'The system shall return analysis results within 30 seconds for documents up to 20 pages' and 'Service 2 shall call Service 3 with a timeout of 10 seconds and return an error if exceeded.'
- Create a requirements traceability matrix linking each requirement to at least one user story.
- Build Figma wireframes for three screens: Upload + Field Confirmation, Error Report Results, Dispute Letter Preview. Annotate each element with its requirement ID.

Step 2 — GitHub, Trello & Environment Setup [Sprint 1 — Member 2]

- Create GitHub repo immediately. Share with quantic-grader. Create branch protection: all merges to main via PR, approved by Code Owner.
- Create Trello Scrum Board with columns: Backlog / Sprint Backlog / In Progress / Done. Populate full product backlog from user stories before writing any code.
- Create Python virtual environment. Pin all dependencies in requirements.txt for both Flask services. Provide README.md with complete setup instructions.
- Create a .env.example file documenting all required environment variables (OpenAI key, Textract credentials, database URL, Service 3 URL) without real values.

Step 3 — Three-Service Infrastructure & CI/CD [Sprint 2 — Member 2]

- Create three Render services from the GitHub repo: one Static Site (Service 1), two Web Services (Services 2 and 3). Each gets its own URL and environment variables on Render.
- Implement /health endpoint in both Flask services returning {status: 'ok', service: 'bill-analysis'} and {status: 'ok', service: 'rag'} respectively.
- Set up GitHub Actions workflow: on every PR and push to main — install dependencies, run pytest, run lint. On passing tests on main, trigger redeploy of all three Render services via Render deploy hooks. Post-deploy step pings both /health endpoints.
- Deploy minimal working shells of all three services in Sprint 2 — not Sprint 1. By Sprint 2 you have requirements and wireframes, so the scaffolding you build is purposeful from the first commit.

Step 4 — OCR & Field Confirmation [Sprint 2 — Member 2 leads]

- Integrate AWS Textract in Service 2. POST /upload accepts PDF, calls Textract AnalyzeDocument, stores raw extraction in PostgreSQL session.
- Build React field-confirmation component in Service 1. Display extracted key-value pairs. User can edit any field. POST /confirm stores confirmed structured data.
- PostgreSQL schema: Sessions, Documents, ExtractedFields, AnalysisResults, DisputeLetters tables. Use SQLAlchemy ORM.

- Apply Flexbox or CSS Grid layout throughout Service 1. Consistent Tailwind CSS theming across all four pages.

Step 5 — Knowledge Base Ingestion & RAG Chain [Sprint 3 — Member 1 leads]

- Download and process all four CMS knowledge base sources. Parse PDFs with pypdf, CSVs with pandas, clean and chunk by heading or token window with overlap.
- Embed chunks with OpenAI text-embedding-3-small. Store in ChromaDB with metadata: source, document title, section, page number.
- Build LangChain LCEL retrieval chain: top-5 retrieval, prompt injection with citations, GPT-4o-mini generation at temperature=0.
- Add guardrails: refuse questions outside the medical billing domain; always include source document IDs in response; limit output to 300 tokens.
- Implement POST /explain and POST /draft-letter endpoints in Service 3. Test independently before Service 2 integration.

Step 6 — Four Error Detection Modules [Sprint 3 — Member 2 leads]

- Implement BaseDetector abstract class with detect(confirmed_fields) → List[DetectionResult] method signature.
- Implement all four detector subclasses. Each returns structured JSON: {module, error_type, description, line_items_affected, estimated_dollar_impact, confidence}.
- Implement ErrorDetectionEngine accepting a list of BaseDetector instances, running all and aggregating results.
- Write unit tests for each module with known fixture data. Minimum 3 tests per module (positive, negative, boundary case).
- Implement POST /analyse in Service 2: runs all four modules, calls Service 3 POST /explain, merges explanations, returns complete results JSON.

Step 7 — RAG Explanations, Results UI & Dispute Letter [Sprint 4 — Both]

- Service 3: wire RAG explanation chain into POST /explain. For each DetectionResult received from Service 2, retrieve relevant knowledge base passages and return a grounded plain-English explanation with source citations.
- Service 2: call Service 3 POST /explain after running all four modules. Merge explanation text and citations into the results JSON payload returned to Service 1.
- Service 1: build error report results page. Colour-coded error cards (red = high impact, amber = medium, green = informational). Show estimated dollar impact per error, RAG-grounded explanation, and regulatory citation reference.
- Service 3: implement POST /draft-letter. Accepts full results payload, generates dispute letter content with specific errors cited, dollar amounts, and regulatory references grounded in the knowledge base.
- Service 2: call Service 3 POST /draft-letter, receive letter content, format via python-docx template and ReportLab PDF. Return both file formats for download.
- Service 1: add download buttons for Word and PDF dispute letter on results page. Full end-to-end flow verified: upload → confirm → analyse → explained report → dispute letter download.

Step 8 — RAG Evaluation [Sprint 5 — Member 1 leads]

- Prepare evaluation set of 15-20 questions drawn from the knowledge base. Write gold answers manually based on the source documents before testing.
- Measure groundedness: for each answer, manually verify that all factual claims are supported by the retrieved context. Report as a percentage.

- Measure citation accuracy: for each citation, verify it points to the specific passage supporting the claim. Report as a percentage.
- Measure latency: time 10-20 queries end-to-end. Report p50 and p95.
- Document results in design-and-evaluation.md in the repo. Include the evaluation question set and gold answers.

Step 9 — Design Documentation [Sprint 5 — Both]

- UML structural diagram: class diagram showing BaseDetector, four detector subclasses, ErrorDetectionEngine, Flask routes, SQLAlchemy models, and RAGChain.
- UML behavioural diagram: sequence diagram showing the full flow from PDF upload through Service 2 → Service 3 → results → letter download.
- DDD bounded context diagram: two bounded contexts (Bill Processing and Knowledge & Explanation), their entities, and the inter-service API at the boundary.
- Microservices architecture diagram: three services, their Render URLs, PostgreSQL database, ChromaDB, and inter-service communication arrows.
- Deployment cost analysis: free-tier prototype (Render + AWS Textract free tier = ~$0/month) vs. production scale (Render paid + RDS + Textract at volume). Include table with cost estimates.
- Document anti-patterns found and removed. At minimum: bare except blocks in Flask routes, global database state, monolithic route handlers without service layer.

Step 10 — Final Testing, Code Freeze & Presentation [Sprint 5 → Sprint 6]

- Sprint 5: Complete test suite — 15+ unit tests across all modules, integration tests for the full pipeline, post-deploy smoke tests for all three /health endpoints. Run pytest --cov and include coverage summary in design document. CODE FREEZE at end of Sprint 5.
- Sprint 5: Prepare 2-3 sample documents with known errors. Verify all produce correct output. Store in /test-data folder.
- Sprint 6 (1 week): Warm up all three Render services. Full demo rehearsal with timer. Record 15-20 minute Zoom presentation — both members on camera, government IDs shown at start, full end-to-end demo, CI/CD pipeline shown running.
- Sprint 6: Upload recording to YouTube unlisted. Submit GitHub repo link + recording link as a single PDF to Quantic dashboard.

SUBMISSION CHECKLIST

#	Deliverable	Owner	Location
1 ★	GitHub repo with all source code, shared with quantic-grader	Member 2	GitHub repo URL
2 ★	Three deployed Render service URLs	Member 2	README.md and deployed.md
3 ★	Trello board with all user stories and sprint completion status	Member 1	Link in README
4 ★	Design and testing document (patterns, UML, DDD, microservices, cost analysis, test results)	Both	design-and-testing.md
5 ★	CI/CD pipeline deploying all three services with automated tests	Member 2	.github/workflows/

6 ★	15-20 min recorded presentation, both members on camera, IDs shown	Both	YouTube unlisted or Google Drive
7	15+ user stories, user journey, 20+ requirements, traceability matrix	Member 1	requirements.md
8	Figma wireframes (3 screens) with requirement ID annotations	Member 1	Figma link in requirements.md
9	UML structural + behavioural diagrams	Member 2	design-and-testing.md
10	DDD bounded context diagram	Both	design-and-testing.md
11	Microservices architecture diagram (3 services, APIs, DB)	Member 2	design-and-testing.md
12	Deployment cost analysis (free tier vs production)	Both	design-and-testing.md
13	RAG evaluation results (groundedness, citation accuracy, latency)	Member 1	design-and-evaluation.md
14	ai-tooling.md documenting AI tools used	Both	ai-tooling.md
15	Sprint demo recordings (6 total — one per sprint; Sprint 6 is presentation recording)	Both	Shared folder or repo wiki

TIPS & RESOURCES

Render Multi-Service Setup

- From your GitHub repo, create three services on Render: one Static Site pointing to your React build command, and two Web Services pointing to your two Flask apps (each in its own subdirectory or started with different entry points).
- Use Render environment groups to share common variables (e.g. OPENAI_API_KEY) across services without duplicating secrets.
- Store the Service 3 URL as an environment variable in Service 2 (e.g. RAG_SERVICE_URL=https://medicheck-rag.onrender.com). Never hardcode service URLs.
- Free tier services cold-start after 15 minutes of inactivity (~30 seconds). For your demo, open the app 2 minutes before recording to warm all three services.

Inter-Service Communication

- Service 2 calls Service 3 using the requests library. Add a 10-second timeout and catch requests.Timeout explicitly — return a graceful error if Service 3 is unavailable rather than crashing Service 2.
- Both services should return a /health endpoint. Your GitHub Actions post-deploy step should ping both after deployment and fail the workflow if either returns non-200.
- Use structured JSON for all inter-service payloads. Define a shared schema in a common models.py file (can be copied to both services) to avoid drift.

Strategy Pattern Implementation

- Define BaseDetector in a detectors/base.py file with an abstract detect(confirmed_fields: dict) -> list[dict] method. Import abc.ABC and abc.abstractmethod.
- Each of the four detector subclasses lives in its own file: detectors/duplicate.py, detectors/medicare_rate.py, detectors/eob_reconciliation.py, detectors/no_surprises.py.
- ErrorDetectionEngine in engine.py accepts detectors: list[BaseDetector] in its constructor. The run() method iterates and aggregates. Document in UML class diagram — this is your primary design pattern evidence.

Testing Strategy

- For duplicate detector tests: fixture with bill containing CPT 29881 on the same date twice (should flag) and on different dates (should not flag).
- For Medicare rate tests: mock the CMS fee schedule lookup. Test at 290% rate (no flag), 310% rate (flag), and exactly 300% boundary.
- For inter-service integration tests: mock Service 3 responses using unittest.mock. Test that Service 2 correctly handles Service 3 being unavailable (timeout scenario).
- For RAG tests: test that POST /explain returns a response containing a 'citations' key when queried with a question answerable from the knowledge base.

Presentation

- Open all three Render URLs before recording. Have the browser tabs pre-loaded. Do not navigate to Render URLs during the recording — cold start on camera will cost you 30-60 seconds.
- Use a pre-prepared sample bill you have tested 10+ times. Know exactly which four errors it contains and what the expected dollar impact is. Never demo with a document you haven't verified in advance.

- For the CI/CD segment: have a GitHub Actions run from your last merge already open in a browser tab. Show the test results, the three deploy steps, and the /health smoke test results. Point to each explicitly.

PROJECT RUBRIC

Scores 3 and above are considered passing. Students who receive a 0-2 will not receive credit and must revise and resubmit.

Score	Description
5	Addresses ALL requirements: • All three Render services deployed and publicly accessible • All four error detection modules working correctly, unit-tested, Strategy pattern documented with UML • RAG pipeline producing grounded, cited responses; evaluation results (groundedness, citation accuracy, latency) documented • Full design and testing document: Strategy + Chain of Responsibility patterns with UML; DDD bounded context diagram; microservices architecture diagram; deployment cost analysis; all anti-patterns resolved • CI/CD pipeline: tests on every PR; auto-deploy of all three services; post-deploy /health smoke test on all services • 15+ user stories from 3 personas; complete user journey; 20+ quantifiable traced requirements; annotated Figma wireframes • Dispute letter generation: Word and PDF download with regulatory citations • Outstanding 15-20 min presentation: all modules demonstrated, CI/CD shown running, both members visible/audible, government IDs shown Shows clear initiative above and beyond minimum requirements.
4	Addresses MOST requirements: • All three services deployed, mostly working • At least three of four detection modules working; Strategy pattern present • RAG pipeline functional with mostly correct citations; evaluation reported with minor gaps • Design document present: at least one pattern with UML; bounded context diagram; microservices diagram • CI/CD present and functional with minor gaps • 10-15 user stories; user journey; 15-20 requirements; at least one wireframe • Dispute letter generation working for at least one format • Clear presentation covering most functionality; both members present Shows good initiative.
3	Addresses SOME requirements: • At least two services deployed and accessible • At least two detection modules working • RAG pipeline partially functional; some correct responses • Design document limited: at least one pattern mentioned; basic diagrams • CI/CD present but incomplete • 10+ user stories; basic requirements; basic wireframe • Some form of output displayed to user Shows some initiative.
2	Addresses FEW requirements:

	• Only one service deployed or significant issues • Only one detection module working • RAG pipeline partially working; limited correct responses • Design document very limited • CI/CD present but largely non-functional • Fewer than 10 user stories; fewer than 10 requirements Shows little initiative.
1	Most components missing: • No working deployment • No functional detection or RAG • No design documentation • No CI/CD • No user stories or requirements • No presentation or severely incomplete
0	Assignment significantly incomplete, plagiarised, or most vital components missing.

PRESENTATION RUBRIC

Scores 2 and above are considered passing.

Score	Description
5	• All user stories demonstrated with correct output • All four modules and RAG explanations shown • Dispute letter downloaded on camera • CI/CD pipeline shown running — test results and three-service deploy visible • Design decisions explained — patterns, CPT licensing, microservices split rationale, cost analysis • Both members presenting substantive content; both visible and audible • Government IDs shown clearly at start • Presentation 15-20 minutes • Screen share clear and legible throughout
4	• Most user stories demonstrated; minor omissions • At least three modules shown • Dispute letter shown • CI/CD shown with minor gaps • Both members visible and audible • IDs shown • Marginally outside 15-20 minute requirement
3	• Majority of user stories shown with some exceptions • At least two modules demonstrated • Both members visible and audible • IDs shown • Significantly outside time requirement

2	• Some user stories shown; several exceptions • One module demonstrated • Both members mostly visible/audible • IDs shown • Significantly outside time requirement
1	• Very limited demo; significant functionality missing • Members not visible or audible • IDs not shown • Significantly outside time requirement
0	Presentation incomplete, plagiarised, or missing most components.

APPENDIX — RECOMMENDED FREE TOOLS

The following tools are recommended for MediCheck development. All have a free tier sufficient for the full 11-week project. Tools are grouped by category.

> **Team note:** Agree on tools as a team in Sprint 1 and document your choices in ai-tooling.md. Consistency matters more than the specific tool — switching halfway through a project is costly.

Project Management & Collaboration

Trello *trello.com* | **Free forever (core features)**
Use: Product backlog, sprint boards, user story tracking. Required by the Capstone rubric.
Tips: Use one board with columns: Backlog / Sprint 1 / Sprint 2 / ... / Done. Add a card per user story. Use labels for: Feature, Bug, Documentation, Testing. Attach the relevant requirement ID to each card description.

Notion *notion.so* | **Free for individuals and small teams**
Use: Meeting notes, sprint retrospective notes, shared documentation drafts, design decisions log.
Tips: Create a shared workspace. Use a simple table for your sprint retrospective notes. Much easier than Google Docs for structured notes that link to each other.

Slack *slack.com* | **Free tier (90-day message history)**
Use: Async team communication, GitHub and Trello notification integrations.
Tips: Connect GitHub to a Slack channel — every PR and CI/CD result posts automatically. Eliminates the need to check GitHub manually.

UI/UX Design & Wireframing

Figma *figma.com* | **Free for up to 3 projects (more than enough)**
Use: Wireframes and mockups for all three core screens. Required by the Project 1 rubric criteria embedded in MediCheck.
Tips: Start with low-fidelity wireframes (grey boxes and labels) before adding colour. Create a component library for your shared UI elements (button, input field, error card) — reuse them across all screens. Share the link publicly in your requirements.md file.

draw.io / diagrams.net *app.diagrams.net* | **Completely free, no account required**
Use: UML diagrams (class + sequence), DDD bounded context diagram, microservices architecture diagram. All deliverables required in the design document.
Tips: Use the UML shape library (View > Shapes > UML). For the microservices diagram, use the AWS or generic network shape library. Export as PNG and embed in your design-and-testing.md document. Save the .drawio source file in the repo under /docs/diagrams/.

Lucidchart *lucidchart.com* | **Free for up to 3 documents**
Use: Alternative to draw.io. Better real-time collaboration but limited free tier.
Tips: If both team members want to edit a diagram simultaneously, Lucidchart handles this better than draw.io. For solo editing, draw.io is simpler and has no limits.

Development Environment & IDE

VS Code *code.visualstudio.com* | **Free, open source**
Use: Primary IDE for both Python (Flask) and JavaScript (React) development.

Tips: Install these extensions: Python (Microsoft), Pylance, ESLint, Prettier, GitLens, REST Client (for testing Flask endpoints without leaving the IDE), and Thunder Client (Postman alternative built into VS Code).

Cursor *cursor.com* | Free tier (2,000 AI completions/month)

Use: AI-assisted coding IDE built on VS Code. Significantly accelerates development — especially useful for boilerplate Flask routes, React components, and test fixtures.

Tips: Load your requirements.md and architecture design into Cursor's context before starting each sprint. The AI suggestions are much more relevant when it understands your domain. Describe your MediCheck architecture once using the @codebase command.

GitHub Copilot *github.com/features/copilot* | Free for verified students (GitHub Education)

Use: AI code completion integrated directly into VS Code. Alternative to Cursor for in-editor AI assistance.

Tips: Apply for GitHub Student Developer Pack if you qualify. Copilot is particularly good for writing pytest fixtures and docstrings — two things that take time but are important for your rubric score.

API Development & Testing

Postman *postman.com* | Free for individual use

Use: Test your Flask API endpoints during development. Test inter-service calls between Service 2 and Service 3.

Tips: Create a MediCheck collection with requests for every endpoint: POST /upload, POST /confirm, POST /analyse, POST /explain, GET /health. Save example request bodies. Share the collection export in the repo under /docs/postman/.

Thunder Client (VS Code Extension) *VS Code marketplace* | Free

Use: Lightweight Postman alternative built directly into VS Code. No separate app needed.

Tips: Good for quick endpoint tests during development. Less powerful than Postman for documentation and sharing, but faster for day-to-day testing.

CI/CD & DevOps

GitHub Actions *github.com/features/actions* | Free for public repos; 2,000 min/month for private repos (more than enough)

Use: CI/CD pipeline: run pytest on every PR, auto-deploy all three Render services on merge to main, post-deploy /health smoke test.

Tips: Store Render deploy hook URLs and API keys as GitHub Actions secrets (Settings > Secrets and variables > Actions). Never commit secrets to the repo. Use a matrix strategy in your workflow to run tests for Python 3.11 — the same version Render uses.

Render *render.com* | Free tier: Static Sites (unlimited), Web Services (750 hrs/month shared across all services)

Use: Deploy all three MediCheck services. Static Site for React, two Web Services for Flask. Render-managed PostgreSQL database.

Tips: 750 free hours/month is enough for three services if they are not all running simultaneously at full capacity. Free tier services spin down after 15 minutes of inactivity — plan for this in your demo. Use the Render deploy hook URL in GitHub Actions to trigger redeployment after tests pass.

AI, LLM & RAG Tools

OpenAI API *platform.openai.com* | $5 free credit on signup; text-embedding-3-small ~$0.02/1M tokens; GPT-4o-mini ~$0.15/1M input tokens

Use: Embeddings for ChromaDB (text-embedding-3-small) and LLM generation for RAG explanations and dispute letters (GPT-4o-mini at temperature=0).

Tips: For an 11-week project with two developers, total LLM costs will likely be under $5. Set a spending limit of $10 in your OpenAI account settings to avoid surprises. Use GPT-4o-mini rather than GPT-4o — it is 10x cheaper and sufficient for structured, grounded output.

OpenRouter *openrouter.ai* | Free tier with rate limits; many free models available

Use: Alternative to OpenAI for LLM generation during development. Useful if you want to test with different models without paying.

Tips: Use OpenRouter's free tier for development and testing. Switch to OpenAI GPT-4o-mini for the final evaluation and demo — consistency matters for your RAG evaluation metrics.

LangChain *python.langchain.com* | Free, open source

Use: RAG pipeline orchestration in Service 3. Handles document loading, chunking, embedding, ChromaDB storage, retrieval chain, and prompt management.

Tips: Use LCEL (LangChain Expression Language) rather than the older chain classes — it is more readable and debuggable. Start with the ConversationalRetrievalChain example in the LangChain docs and adapt it for your medical billing domain.

ChromaDB *trychroma.com* | Free, open source

Use: Local vector store for Service 3. Stores embedded knowledge base chunks for retrieval.

Tips: Use PersistentClient with a local directory path. The ChromaDB directory can be committed to the repo or regenerated from the ingest.py script on first run. For Render deployment, use a Render persistent disk (free 1GB) to store the ChromaDB directory between restarts.

Documentation & Knowledge Base

MkDocs *mkdocs.org* | Free, open source

Use: Optional: generate a documentation website from your Markdown files. Makes design-and-testing.md and requirements.md look professional.

Tips: Not required for the rubric but takes 30 minutes to set up and makes your GitHub repo look significantly more polished to a grader or potential employer.

Mermaid (in Markdown) *mermaid.js.org* | Free — renders in GitHub Markdown automatically

Use: Embed sequence diagrams and flowcharts directly in your Markdown documentation files using code blocks.

Tips: GitHub renders Mermaid diagrams natively in .md files. Use this for your sequence diagram in design-and-testing.md — it is version-controlled alongside the code and always up to date.

Presentation Recording

Zoom *zoom.us* | Free tier (40-minute limit on group calls — record locally before time limit)

Use: Record the final 15-20 minute capstone presentation. Both members join on video. One member shares their screen showing the live app.

Tips: Use 'Record to this computer' rather than cloud recording for better quality. Turn off Zoom's noise suppression (Audio > Original Sound) so the app's audio output is captured cleanly. Start recording before the 40-minute free tier limit — the recording continues even if the call would normally end.

Loom *loom.com* | Free tier: up to 5 minutes per video (not sufficient for final presentation, but good for sprint demos)

Use: Ideal for the sprint demo recordings submitted to the Product Owner at the end of each sprint — these only need to be 3-5 minutes.

Tips: Use Loom for Sprint 1-5 demo recordings (3-5 minutes each). Use Zoom for the Sprint 6 final 15-20 minute capstone presentation.

YouTube (unlisted) *youtube.com* | **Free**

Use: Host the final presentation recording as an unlisted YouTube video. Submit the link to Quantic.

Tips: Upload as unlisted (not private — private videos require the grader to have a Google account linked to your sharing list). Unlisted means anyone with the link can watch. Verify the link works in an incognito browser before submitting.

MediCheck is the full programme in one project — requirements, design, architecture, ML, AI engineering, and agile delivery — built as a properly bounded microservices system with real-world financial impact. These tools are your workbench. Use them well.